

Formal Languages and Compilers
Prof. Breveglieri, Morzenti, Agosta
Written exam: laboratory question

05/09/2022

Time: 60 minutes. Textbooks and notes can be used. Pencil writing is allowed.

Important: Write your name on any additional sheet.

SURNAME (Cognome):

NAME (Nome):

Matricola:or Person Code:

Instructor: ☐ Prof. Breviglieri ☐ Prof. Morzenti ☐ Prof. Agosta

The laboratory question must be answered taking into account the implementation of the Acse compiler given with the exam text.

Modify the specification of the lexical analyser (`flex` input) and the syntactic analyser (`bison` input) and any other source file required to extend the Lance language with the ability of generating pseudo-random numbers.

The implementation must define the **randomize** statement and the **random** operator. When `random` appears in an expression, it generates a new random number from the last one returned (denoted n in the following) using the following formula:

$$(1664525 \times n + 1013904223) \bmod 2^{32}$$

The formula above is called a *Linear Congruential Generator* and the initial value of n is set to 12345. Since all integers in ACSE are 32-bit in size, there is no need to explicitly compute the modulo operator. The `randomize` statement resets n to the value of the expression enclosed between the parenthesis of the statement.

The following code snippet exemplifies the operation of `random` and `randomize`. In the loop at the beginning of the example, the first three random numbers from the sequence are extracted and printed. These numbers are 87628868, 71072467 and -1962130922, respectively. Indeed, $(1664525 \times 12345 + 1013904223) \bmod 2^{32} = 87628868$. The other two numbers are obtained in the same way, but replacing 12345 with 87628868 and 71072467 respectively. After the loop, the `randomize` statement is used to reset n to the value of the expression $12300 + 45$, which is equal to 12345. Since n now is equal to its initial value, the subsequent use of `random` returns the first number in the sequence again, 87628868. Finally, `random` is used inside an expression. The current value of `random` is 71072467; this value is then divided by 1000 and added to 7, resulting in the value 71079 which is then assigned to the variable a . The value of a is then printed to the output.

```
int a, i;

i = 0;
while (i < 3) {
    write(random());          // 87628868, 71072467, -1962130922
    i = i + 1;
}

randomize(12300 + 45);
write(random());             // writes 87628868
a = random() / 1000 + 7;
write(a);                   // writes 71072467 / 1000 + 7 = 71079
```

1. Define the tokens (and the related declarations in **Acse.lex** and **Acse.y**). (3 points)
2. Define the syntactic rules or the modifications required to the existing ones. (4 points)
3. Define the semantic actions needed to implement the required functionality. (18 points)

Solution

The solution is shown below in *diff* format. All lines that begin with '+' were added, while the lines that begin with '-' were removed. **It is not required and it is not encouraged to provide a solution in *diff* format to get the maximum grade.**

```
diff --git a/acse/Acse.lex b/acse/Acse.lex
index 35f9b73..bd35ee0 100644
--- a/acse/Acse.lex
+++ b/acse/Acse.lex
@@ -93,6 +93,8 @@ ID      [a-zA-Z_][a-zA-Z0-9_]*
"return"      { return RETURN; }
"read"        { return READ; }
"write"       { return WRITE; }
+"random"     { return RANDOM; }
+"randomize"  { return RANDOMIZE; }

{ID}          { yylval.svalue=strdup(yytext); return IDENTIFIER; }
{DIGIT}+      { yylval.intval = atoi( yytext ); }
diff --git a/acse/Acse.y b/acse/Acse.y
index 0636dc6..c1909a2 100644
--- a/acse/Acse.y
+++ b/acse/Acse.y
@@ -90,6 +90,9 @@ t_reg_allocator *RA;
        ↪ /* Register allocator. It implements the "Linear

t_io_infos *file_infos;    /* input and output files used by the compiler */

+/* Register that at runtime will contain the last pseudo-random number
+ * generated. */
+int r_random;

extern int yylex(void);
extern void yyerror(const char*);
@@ -125,6 +128,7 @@ extern void yyerror(const char*);
%token RETURN
%token READ
%token WRITE
+%token RANDOM RANDOMIZE

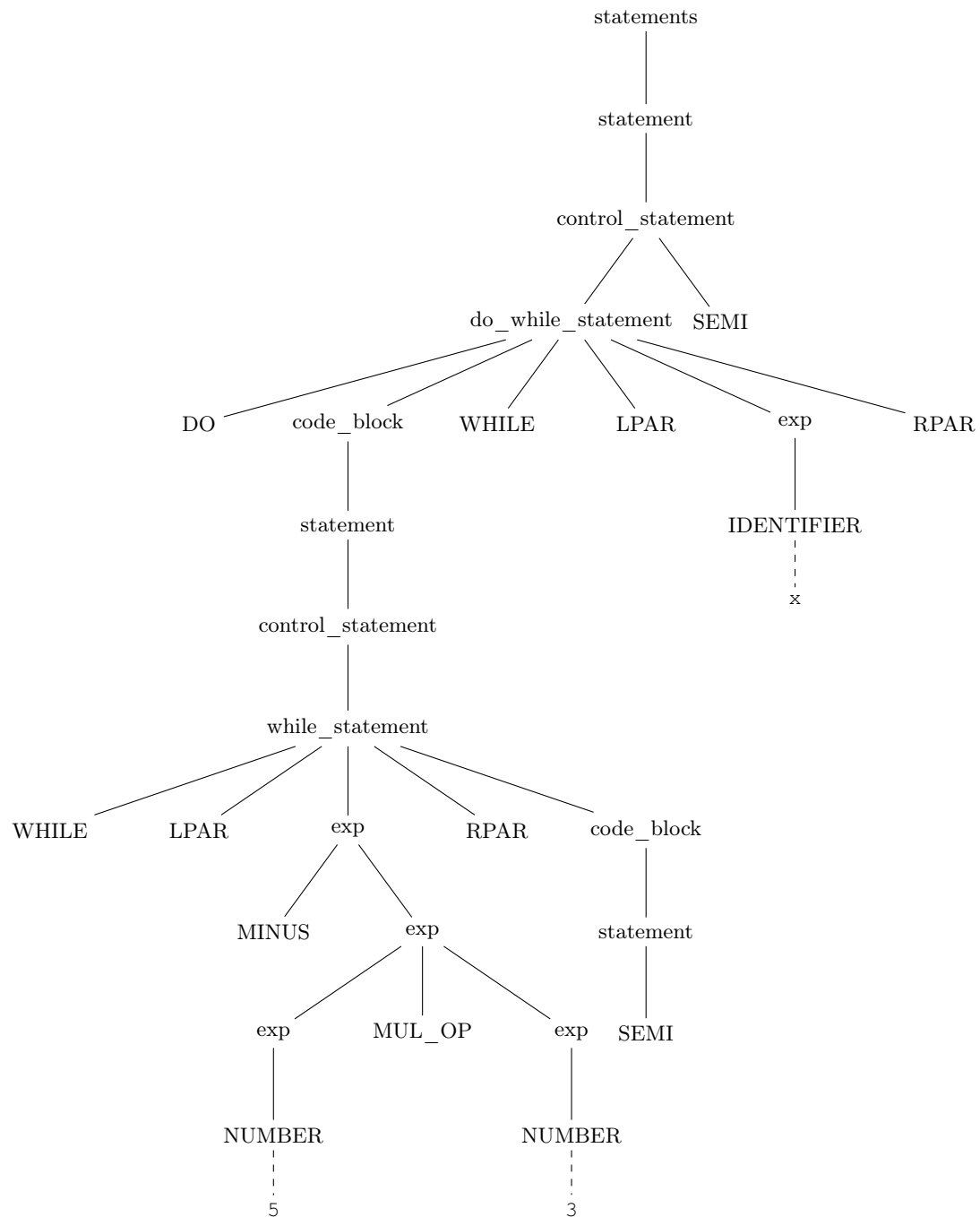
%token <label> DO
%token <while_stmt> WHILE
@@ -167,7 +171,14 @@ extern void yyerror(const char*);
    2. A list of instructions. (at least one instruction!).
    * When the rule associated with the non-terminal 'program' is executed,
    * the parser notifies it to the 'program' singleton instance. */
-program : var_declarations statements EOF_TOK
+program : var_declarations
+{
+    /* Reserve the register used by random() and randomize().
+    * Before the beginning of the program, generate code to initialize
+    * it with its initial value, which is 12345. */
+    r_random = gen_load_immediate(program, 12345);
+}
+statements EOF_TOK
{
    /* Notify the end of the program. Once called
    * the function 'set_end_program' - if necessary -
@@ -247,6 +258,7 @@ statements : statements statement { /* does nothing */ }
statement : assign_statement SEMI { /* does nothing */ }
          | control_statement     { /* does nothing */ }
```


4. Given the following Lance code snippet:

```
do while (-5 * 3); while (x);
```

write down the syntactic tree generated during the parsing with the Bison grammar described in *Acse.y* starting from the statements *nonterminal*. (5 points)

Solution



5. (**Bonus**) Describe how to modify the given implementation of the `random` operator to make it produce pseudo-random numbers in a given range. For example, `random(2, 10)` shall only produce random numbers greater or equal than 2, and lesser than 10.